

**CENTRO UNIVERSITÁRIO FIAP**  
Graduação em Análise e Desenvolvimento de Sistemas

**CHRONOSGEO**  
**Sistema de Monitoramento Ambiental via Satélite**

API REST — Domain Driven Design Using Java  
Framework: Quarkus 3.8.4 | Java 17 | Oracle Database

Global Solution 2026/1

**Millena dos Santos Pamio**  
RM: 566664 | Turma: 1TDSPS

São Paulo — 2026

# SUMÁRIO

| Seção | Título                        | Página |
|-------|-------------------------------|--------|
| 1     | Objetivo e Escopo do Projeto  | 3      |
| 2     | Descrição da Solução Proposta | 3      |
| 3     | Funcionalidades Implementadas | 4      |
| 4     | Tabela de Endpoints           | 5      |
| 5     | Diagrama de Camadas           | 6      |
| 6     | Instruções de Execução        | 7      |
| 7     | Deploy da Aplicação           | 7      |

# 1 OBJETIVO E ESCOPO DO PROJETO

O ChronosGeo é uma API REST desenvolvida com o framework Quarkus e Java 17, projetada para dar suporte ao sistema de monitoramento ambiental via satélite da Global Solution 2026/1. O objetivo é fornecer um backend robusto, escalável e integrado ao banco de dados Oracle, expondo endpoints que permitem ao Front-End (React + Vite + TypeScript) realizar operações completas de CRUD sobre os dados de ocorrências ambientais, alertas e usuários do sistema.

O escopo contempla o gerenciamento das entidades principais mapeadas nas tabelas GS\_OCORRENCIA, GS\_ALERTA e GS\_USUARIO do banco de dados relacional desenvolvido na disciplina de Building Relational Database, garantindo consistência e integridade dos dados através de validações e regras de negócio implementadas na camada Service.

# 2 DESCRIÇÃO DA SOLUÇÃO PROPOSTA

A solução adota a arquitetura em camadas (MVC / DAO), separando responsabilidades entre:

- **Entity:** Classes JPA que mapeiam as tabelas GS\_\* do Oracle com anotações @Entity, @Table, @Column, @ManyToOne e validações Bean Validation.
- **Repository (DAO):** Interfaces que estendem PanacheRepository, fornecendo CRUD básico e métodos de consulta customizados como findByStatus, findByNivel e findByEmail.
- **Service (BO):** Camada de regras de negócio. Valida consistência dos dados (ex.: não permite alerta em ocorrência encerrada, email único, datas coerentes) antes de delegar ao repositório.
- **Resource:** Endpoints JAX-RS que recebem requisições HTTP, delegam à camada Service e retornam respostas JSON com os status HTTP adequados (200, 201, 204, 400, 404, 409, 422, 500).
- **Exception:** Exceção customizada ChronosGeoException e GlobalExceptionHandler que convertem qualquer exceção em resposta JSON padronizada.

O CORS está configurado globalmente via application.properties permitindo que o Front-End hospedado na Vercel consuma a API sem bloqueio de origem.

# 3 FUNCIONALIDADES IMPLEMENTADAS

## 3.1 OcorrenciaResource

| Método                     | Endpoint                  | Descrição                                                                |
|----------------------------|---------------------------|--------------------------------------------------------------------------|
| listar(status, severidade) | GET /api/ocorrencias      | Lista todas as ocorrências ou filtra por status (ATIVA/MONITORAND        |
| buscarPorId(id)            | GET /api/ocorrencias/{id} | Retorna ocorrência pelo ID. Lança 404 se não encontrada.                 |
| criar(ocorrencia)          | POST /api/ocorrencias     | Cria nova ocorrência. Valida: data não futura, datas coerentes, região e |

|                           |                              |                                                                 |
|---------------------------|------------------------------|-----------------------------------------------------------------|
| atualizar(id, ocorrencia) | PUT /api/ocorrencias/{id}    | Atualiza todos os campos de uma ocorrência existente.           |
| deletar(id)               | DELETE /api/ocorrencias/{id} | Remove ocorrência. Regra: só permite remover se ENCERRADA (409) |

### 3.2 AlertaResource

| Método                      | Endpoint                 | Descrição                                                            |
|-----------------------------|--------------------------|----------------------------------------------------------------------|
| listar(nivel, idOcorrencia) | GET /api/alertas         | Lista alertas ou filtra por nível (ATENCAO/ALERTA/PERIGO/EMERGENCIA) |
| buscarPorId(id)             | GET /api/alertas/{id}    | Retorna alerta pelo ID. Lança 404 se não encontrado.                 |
| criar(alerta)               | POST /api/alertas        | Emite alerta. Regra: ocorrência não pode estar ENCERRADA (409).      |
| atualizar(id, alerta)       | PUT /api/alertas/{id}    | Atualiza nível, órgão, descrição e status do alerta.                 |
| deletar(id)                 | DELETE /api/alertas/{id} | Remove alerta fisicamente. Retorna 204.                              |

### 3.3 UsuarioResource

| Método                 | Endpoint                           | Descrição                                                                    |
|------------------------|------------------------------------|------------------------------------------------------------------------------|
| listar(perfil)         | GET /api/usuarios                  | Lista todos os usuários ou filtra por perfil (ANALISTA/GESTOR/ADMINISTRADOR) |
| listarAtivos()         | GET /api/usuarios/ativos           | Lista apenas usuários com ativo = 'S'.                                       |
| buscarPorId(id)        | GET /api/usuarios/{id}             | Busca usuário pelo ID. Retorna 404 se não encontrado.                        |
| criar(usuario)         | POST /api/usuarios                 | Cadastra usuário. Valida unicidade do email (409 se duplicado).              |
| atualizar(id, usuario) | PUT /api/usuarios/{id}             | Atualiza nome, email, perfil e órgão. Valida email duplicado.                |
| desativar(id)          | PATCH /api/usuarios/{id}/desativar | Soft delete: seta ativo='N'. Retorna 409 se já inativo.                      |
| deletar(id)            | DELETE /api/usuarios/{id}          | Remove fisicamente o usuário. Retorna 204.                                   |

## 4 TABELA DE ENDPOINTS — API REST

| Verbo  | URI                             | Status Sucesso | Status Erro |
|--------|---------------------------------|----------------|-------------|
| GET    | /api/ocorrencias                | 200 OK         | 400         |
| GET    | /api/ocorrencias/{id}           | 200 OK         | 404         |
| GET    | /api/ocorrencias?status=...     | 200 OK         | 400         |
| GET    | /api/ocorrencias?severidade=... | 200 OK         | 400         |
| POST   | /api/ocorrencias                | 201 Created    | 400/422     |
| PUT    | /api/ocorrencias/{id}           | 200 OK         | 404/422     |
| DELETE | /api/ocorrencias/{id}           | 204 No Cont.   | 404/409     |
| GET    | /api/alertas                    | 200 OK         | —           |
| GET    | /api/alertas/{id}               | 200 OK         | 404         |
| GET    | /api/alertas?nivel=...          | 200 OK         | 400         |
| GET    | /api/alertas?idOcorrencia=...   | 200 OK         | 404         |
| POST   | /api/alertas                    | 201 Created    | 400/409/422 |
| PUT    | /api/alertas/{id}               | 200 OK         | 404         |
| DELETE | /api/alertas/{id}               | 204 No Cont.   | 404         |
| GET    | /api/usuarios                   | 200 OK         | —           |
| GET    | /api/usuarios/ativos            | 200 OK         | —           |
| GET    | /api/usuarios/{id}              | 200 OK         | 404         |
| GET    | /api/usuarios?perfil=...        | 200 OK         | —           |
| POST   | /api/usuarios                   | 201 Created    | 409/422     |
| PUT    | /api/usuarios/{id}              | 200 OK         | 404/409     |
| PATCH  | /api/usuarios/{id}/desativar    | 200 OK         | 404/409     |
| DELETE | /api/usuarios/{id}              | 204 No Cont.   | 404         |

## 5 ARQUITETURA EM CAMADAS

O projeto segue o padrão MVC com separação em camadas, conforme descrito no diagrama textual abaixo:

| Camada           | Classes                                                   | Responsabilidade                                                         |
|------------------|-----------------------------------------------------------|--------------------------------------------------------------------------|
| Entity           | Satelite, Regiao, TipoEvento, Ocorrencia, Alerta, Usuario | Mapeamento JPA das tabelas GS_* do Oracle                                |
| Repository (DAO) | OcorrenciaRepository, AlertaRepository, UsuarioRepository | Acesso ao banco via PanacheRepository. Métodos de consulta customizados. |
| Service (BO)     | OcorrenciaService, AlertaService, UsuarioService          | Regras de negócio, validações, orquestração entre entidades.             |
| Resource         | OcorrenciaResource, AlertaResource, UsuarioResource       | Endpoints JAX-RS REST. Recebe HTTP, delega ao Service, retorna JSON.     |
| Exception        | ChronosGeoException, GlobalExceptionHandler               | Exceção customizada e mapeamento de erros para JSON padronizado.         |
| Config           | application.properties                                    | Datasource Oracle, CORS, JPA, porta HTTP, logging.                       |

Fluxo de uma requisição: Front-End → Resource → Service → Repository → Oracle DB → Response JSON ao Front-End.

## 6 INSTRUÇÕES DE EXECUÇÃO

---

### 6.1 Ferramentas e Versões Necessárias

- **Java JDK:** 17 (LTS) ou superior — [adoptium.net](https://adoptium.net)
- **Maven:** 3.9.x ou superior — [maven.apache.org](https://maven.apache.org)
- **IDE:** IntelliJ IDEA, Eclipse IDE for Enterprise Java, ou VS Code + Extension Pack for Java
- **Banco:** Oracle Database (credenciais fornecidas pelo professor da FIAP)
- **Git:** Para clonar o repositório — [git-scm.com](https://git-scm.com)

### 6.2 Configurar Credenciais

Edite o arquivo `src/main/resources/application.properties` e ajuste:

```
quarkus.datasource.username=
quarkus.datasource.password=
quarkus.datasource.jdbc.url=jdbc:oracle:thin:@:1521:orcl
```

### 6.3 Executar em Desenvolvimento

```
mvn quarkus:dev
```

A API ficará disponível em `http://localhost:8080/api/` com hot reload ativado.

### 6.4 Importar no IntelliJ IDEA

1. File → Open → selecionar a pasta raiz do projeto (onde está o `pom.xml`)
2. Aguardar o Maven baixar as dependências automaticamente
3. Verificar que o JDK 17 está configurado em Project Structure (Ctrl+Alt+Shift+S)
4. Executar a classe `main` ou usar o plugin Quarkus Run Configuration

## 7 DEPLOY DA APLICAÇÃO

---

O deploy deve ser realizado em um servidor de aplicação na nuvem (`render.com`, `railway.app` ou equivalente). Passos resumidos:

1. Gerar o build: `mvn clean package -DskipTests`
2. O artefato será gerado em `target/quarkus-app/`
3. Configurar as variáveis de ambiente no servidor (`datasource`, credenciais Oracle)
4. Iniciar com: `java -jar target/quarkus-app/quarkus-run.jar`
5. Inserir a URL pública do deploy no `integrantes.txt` e no README do GitHub

Importante: o Front-End (React + Vite) deve consumir a URL pública da API configurada neste deploy. O CORS está habilitado para todas as origens (`quarkus.http.cors.origins=*`) para permitir a integração.

